PROJECT BOOK · CAHIER DES CHARGES

iCub, the Little Gardener

A robot-learning simulation with liquids: from a blank Ubuntu laptop to imitation learning and a Vision-Language-Action model

What this book is. A complete specification and learning guide for building your own simulated robot-learning project: the iCub humanoid robot picks up a small watering can with its five-fingered hand and waters plants, with the water simulated as particles. On top of the simulation you will generate expert demonstrations automatically, train an imitation-learning policy (ACT), and fine-tune a small Vision-Language-Action model (SmolVLA) so the robot follows spoken-style instructions like *“water the left plant, just a little”*.

Stack. threepp (3D scene, rendering, PhysX physics, RL tools) · NVIDIA PhysX · NVIDIA Warp · NVIDIA cuRobo (motion planning) · PyTorch · Hugging Face LeRobot (datasets, ACT, SmolVLA).

Hardware it is written for. Laptop with Intel Core i5-12500H, 16 GB RAM, NVIDIA RTX 3050 Laptop GPU (4 GB VRAM), Ubuntu 22.04 dual-boot, plus free cloud GPUs (Kaggle, Lightning AI) for the heavy training.

Version	Date	Status
1.0	2 October 2026	Setup phase in progress: NVIDIA driver working, Python environment being created

Facts about tools, versions and free-tier quotas were checked on 2 October 2026. Software in this field changes monthly, so when a command in this book disagrees with a tool's official README, trust the README and note the change in your project log.

Contents

How to use this book . . . 5

1. Project charter . . . 6

1.1 Context . . . 6

1.2 Vision . . . 6

1.3 Objectives . . . 6

1.4 Scope . . . 7

1.5 Deliverables . . . 7

1.6 Constraints . . . 7

1.7 Requirements . . . 8

1.8 Acceptance criteria (definition of done) . . . 8

2. System architecture . . . 9

2.1 Components . . . 9

2.2 Data flow and files . . . 9

2.3 Where each stage runs . . . 9

3. Hardware and compute strategy . . . 11

3.1 Your machine . . . 11

3.2 VRAM budget (approximate) . . . 11

3.3 The hybrid plan (free) . . . 11

3.4 Laptop habits for long runs . . . 12

4. The software stack explained . . . 13

4.1 The base: Ubuntu, NVIDIA driver, CUDA . . . 13

4.2 Conda environments . . . 13

4.3 PyTorch . . . 13

4.4 threepp . . . 13

4.5 NVIDIA PhysX . . . 14

4.6 NVIDIA Warp . . . 14

4.7 NVIDIA cuRobo . . . 15

4.8 Hugging Face LeRobot . . . 15

4.9 The iCub models package . . . 16

4.10 Git, GitHub and Hugging Face Hub . . . 16

5. Knowledge primer . . . 17

5.1 Linux and the terminal . . . 17

5.2 GPUs, CUDA and memory . . . 17

5.3 Describing a robot: URDF . . . 18

5.4 Kinematics . . . 18

5.5 Controlling joints: PD control . . . 19

5.6 Rigid-body simulation and contacts . . . 19
5.7 Simulating water with particles . . . 19
5.8 Motion planning . . . 20
5.9 Grasping with a multi-fingered hand . . . 20
5.10 Cameras and rendering . . . 21
5.11 Imitation learning . . . 21
5.12 Vision-Language-Action models (VLAs) . . . 22
5.13 Robot datasets: the LeRobot format . . . 22
5.14 Domain randomisation and sim-to-real . . . 23
5.15 Evaluating honestly . . . 23
5.16 Reinforcement learning (optional extension) . . . 23
6. The robot: iCub . . . 24
6.1 Which model file to use . . . 24
6.2 What you build around iCub . . . 25
7. Task specification: the Little Gardener . . . 26
7.1 The scene . . . 26
7.2 The episode, step by step . . . 26
7.3 Instructions (language) . . . 26
7.4 Randomisation ranges (starting values) . . . 26
7.5 Observation and action spaces . . . 27
7.6 Success metrics . . . 27
7.7 Dataset target . . . 27
8. Step-by-step build plan . . . 28
Phase 0 — Environment setup (week 1) . . . 28
Phase 1 — iCub in the simulator (week 2) . . . 29
Phase 2 — The garden scene and water (week 3) . . . 29
Phase 3 — The scripted expert and demo generation (weeks 4–5) . . . 30
Phase 4 — Imitation learning with ACT (week 6) . . . 31
Phase 5 — Fine-tuning SmolVLA (weeks 7–8) . . . 31
Phase 6 — Showcase (week 9) . . . 31
9. Schedule . . . 32
9.1 A typical 5-hour day . . . 32
10. Risks and mitigations . . . 33
11. Repository and engineering practices . . . 34
11.1 Suggested layout . . . 34
11.2 Practices . . . 34
11.3 Daily log template (docs/log.md) . . . 34
12. Troubleshooting journal . . . 35

12.1 Safety nets you now have . . . 35
12.2 Diagnostic commands . . . 35
13. Showcase and career value . . . 37
13.1 What to publish . . . 37
13.2 Results table template . . . 37
13.3 Skills this project demonstrates . . . 37
13.4 Possible extensions (version 2) . . . 37
Appendix A — Glossary . . . 38
Appendix B — Resources and links . . . 40
Appendix C — Command cheat sheet . . . 42

START HERE

How to use this book

The book has three kinds of content, and you will use them differently:

- **The specification (Chapters 1–3, 6–7).** What you are building, why, the constraints, and how you will know it is finished. Read these once fully, then come back when you have to make a design decision.
- **The knowledge primer (Chapters 4–5).** Every concept the project needs, explained from the ground up, with links to go deeper. You do not need to read it all before starting. Each phase of the plan tells you which sections to read first.
- **The execution plan (Chapters 8–12).** Step-by-step phases, the 9-week schedule, risks, the repository layout, and a troubleshooting journal. This is the part you will have open every day.

Throughout the book you will see coloured boxes:

> **EXPLAINER · concept explained**
> Blue boxes explain something new or complicated in plain words.

> **TIP · practical advice**
> Green boxes are shortcuts and habits that save time.

> **WATCH OUT · common trap**
> Orange boxes flag the mistakes that cost people days.

Reading order for week 1: Chapter 1 (the project), Chapter 3 (your hardware), Section 5.1–5.2 (Linux and GPUs), then Phase 0 in Chapter 8.

SPECIFICATION

1. Project charter

1.1 Context

Robot learning on tasks with liquids is hard to try at home. NVIDIA Isaac Sim needs a high-end GPU (16 GB class), its particle fluids are limited, and MuJoCo, the other popular simulator, has no real liquid simulation. A recent public demo showed that a lighter stack — **threepp + PhysX + NVIDIA Warp + cuRobo** — can run a full manipulation scene with particle water on a 6 GB laptop GPU: an arm unscrews a tube, pours it into a beaker and puts it back, holding the tube by friction only.

This project uses the same stack but builds something new: a different robot (iCub with articulated fingers), a different task (watering plants), and a learning pipeline on top (imitation learning and a Vision-Language-Action model). It is a personal portfolio project aimed at industry roles in robotics, embodied AI and computer vision.

1.2 Vision

> **TIP · one-sentence pitch**
>
> *“A child-sized humanoid robot that waters its plants on command — trained entirely in simulation on a 4 GB laptop GPU, using automatically generated demonstrations and a small Vision-Language-Action model.”*

1.3 Objectives

ID	Objective	Measured by
O1	Build a stable simulation: iCub torso, right arm and right hand, a table, a toy watering can, two plant pots, particle water.	Scene runs 60 s without explosions or jitter; water stays in the can when still.
O2	Create a scripted expert that waters a chosen plant using cuRobo for the arm and a scripted grasp for the hand.	Expert success rate of at least 80% over 50 randomised scenes.
O3	Generate a dataset of demonstrations in LeRobot format with images, joint states, actions and a language instruction.	200–500 successful episodes, uploaded to Hugging Face Hub.
O4	Train an imitation-learning policy (ACT) from images.	Success rate in simulation, compared with the expert.
O5	Fine-tune SmolVLA so it follows instructions (which plant, how much water).	Success rate and instruction-following accuracy on held-out scenes.
O6	Publish the project: ray-traced video, README, results table, post.	Public GitHub repo + video + write-up.

1.4 Scope

In scope

- One robot: iCub, upper body only, fixed base. Right arm (7 joints), torso (up to 3 joints), right hand.
- One task family: grasp the watering can by its handle, lift, pour into a target pot, put the can back.
- Particle-based water (a few thousand particles), counted per pot to measure success.
- Automatic demonstrations (cuRobo plans the arm, scripted hand synergy closes the fingers).
- Two learned policies: ACT (local) and SmolVLA (fine-tuned on a free cloud GPU).
- Evaluation entirely in simulation.

Out of scope (for version 1)

- Walking, legs, balance, both arms at once.
- Real robot hardware and sim-to-real transfer (discussed in 5.14 as a future step).
- Physically exact fluid behaviour (viscosity, surface tension). Water is an approximation.
- Large VLAs (OpenVLA 7B, π0): they need far more GPU memory than is available for free.
- Teleoperation with VR or gloves (demos are generated automatically).

1.5 Deliverables

#	Deliverable	Format
D1	Working environment on the laptop (driver, conda env, stack installed)	Install notes in the repo
D2	Simulation scene (iCub + garden + water)	Python package in the repo
D3	Scripted expert and data-generation script	Python scripts + config files
D4	Demonstration dataset	LeRobot dataset on Hugging Face Hub
D5	Trained ACT checkpoint and evaluation report	Checkpoint + results table
D6	Fine-tuned SmolVLA checkpoint and evaluation report	Checkpoint + results table
D7	Showcase video (ray-traced), README, post	MP4 + Markdown + LinkedIn

1.6 Constraints

Constraint	Value	Consequence
GPU memory	4 GB (RTX 3050 Laptop), ~300 MB used by the desktop	One heavy GPU job at a time; small scenes; small images
System RAM	16 GB	Close the browser during heavy runs
Budget	Free	Kaggle (about 30 GPU h/week) and Lightning AI free credits for training
Internet	~200–350 KB/s	Large downloads take hours; download once, cache, avoid re-installing
Time	~5 h/day, ~30–35 h/week	About 9 weeks for version 1

Constraint	Value	Consequence
OS	Ubuntu 22.04 (dual-boot with Windows)	cuRobo and Warp are best supported on Linux

1.7 Requirements

Functional requirements

ID	Requirement
FR1	The scene loads iCub from its official URDF with movable finger joints.
FR2	The watering can is held by contact and friction only (no welding/attaching to the hand).
FR3	Water is simulated as particles and can be poured; the number of particles in each pot and on the table can be counted.
FR4	Scene parameters (can pose, plant positions, water amount, target, lighting) are randomised from a config file with a seed.
FR5	The expert produces a full episode from a language instruction and a scene seed.
FR6	Every episode records: camera image(s), joint positions, actions, instruction text, success flag.
FR7	A policy (ACT or SmolVLA) can be plugged in to replace the expert and evaluated on N scenes.
FR8	A rendering mode produces a high-quality (ray-traced) video of any episode.

Non-functional requirements

ID	Requirement
NFR1	Peak GPU memory of any single stage stays under ~3.5 GB on the laptop.
NFR2	Everything is reproducible: same seed and config give the same scene and the same expert episode.
NFR3	Data generation can run unattended overnight and resumes after a crash without losing finished episodes.
NFR4	Code is versioned on GitHub; datasets and checkpoints on Hugging Face Hub.
NFR5	A newcomer can install and run the demo by following the README.

1.8 Acceptance criteria (definition of done)

1. The scripted gardener video exists: iCub grasps the can by friction, waters the named plant, puts the can back.
2. Dataset of at least 200 successful episodes published, with a dataset card.
3. ACT reaches a measurable success rate on 50 held-out scenes (any honest number is acceptable; the comparison matters).
4. SmolVLA is evaluated on the same 50 scenes, plus an instruction-following test where only the instruction changes.
5. The repo README contains install steps, a results table, the video and known limitations.

SPECIFICATION

2. System architecture

The system is a pipeline of five stages. The crucial design choice for a 4 GB GPU is that the stages run **one after another**, each writing files that the next one reads, so two heavy GPU programs never share memory.

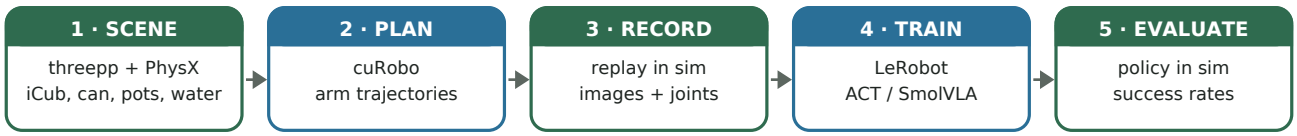


Figure 2.1 — The staged pipeline. Each arrow is a set of files on disk, not a live connection.

2.1 Components

Component	Tool	Job in this project
Scene and rendering	threepp (Python bindings)	Scene graph, cameras, headless render-to-NumPy for datasets, ray-traced video
Rigid bodies and robot	PhysX (via threepp)	iCub articulation, joint drives, contacts, friction, the can and pots
Water	Particles (PhysX particles or a Warp kernel)	Pouring; counting particles per pot
Custom GPU code	NVIDIA Warp	Fast particle counting, custom physics or reward kernels
Arm motion planning	NVIDIA cuRobo	Collision-free arm trajectories to pre-grasp, pour and place poses
Hand control	Your script	Grasp synergy: open → wrap → squeeze
Learning	PyTorch + LeRobot	Dataset format, ACT and SmolVLA training/inference
Storage and sharing	GitHub + Hugging Face Hub	Code, datasets, checkpoints

2.2 Data flow and files

- **configs/scene.yaml** — randomisation ranges, particle count, camera settings.
- **data/plans/episode_00042.npz** — cuRobo output: joint trajectory for the arm + hand commands + metadata.
- **data/lerobot/gardener_v1/** — the recorded dataset (Parquet tables + MP4 videos), pushed to Hugging Face.
- **outputs/train/act_v1/** — checkpoints and training logs.
- **results/eval_act_v1.json** — per-scene success, water in target, spills, instruction.

2.3 Where each stage runs

Stage	Laptop (RTX 3050, 4 GB)	Kaggle / Lightning (16 GB T4/P100)
Scene building and debugging	Yes (main place)	No
cuRobo planning	Yes, small batch sizes	Possible but not needed

Stage	Laptop (RTX 3050, 4 GB)	Kaggle / Lightning (16 GB T4/P100)
Recording demos	Yes, overnight	Optional, if too slow
ACT training	Short test runs	Full runs
SmolVLA fine-tuning	Only if it fits (tight)	Yes (main place)
Evaluation in sim	Yes	No (needs the sim)
Ray-traced video	Yes, offline frame by frame	Optional

SPECIFICATION

3. Hardware and compute strategy

3.1 Your machine

Part	Spec	Verdict
CPU	Intel Core i5-12500H, 12 cores / 16 threads	Good: CPU physics and data processing are fine
RAM	16 GB (about 70% used when idle in Windows)	Enough; close Chrome during heavy runs
GPU 0	NVIDIA RTX 3050 Laptop, 4 GB, driver 595.91.07, CUDA 13.2	The bottleneck. Ampere generation, so cuRobo supports it
GPU 1	Intel Iris Xe (integrated)	Ignore for compute; can run the desktop
Disk	~70 GB free on the Ubuntu partition	OK; keep 20 GB free for datasets and caches

EXPLAINER · what VRAM is and why 4 GB is the limit

VRAM is the memory on the graphics card. Anything the GPU computes on — physics particles, neural-network weights, images being rendered — must sit in VRAM. Unlike normal RAM, there is no comfortable “swap”: when VRAM is full, the program crashes with *CUDA out of memory*. 4 GB is enough for one small job at a time, which is why the pipeline is staged.

3.2 VRAM budget (approximate)

Item	Rough VRAM	Notes
Ubuntu desktop (nvidia mode)	~0.3 GB	Switch to on-demand mode later to free it
CUDA context per process	~0.3–0.5 GB	Every program that uses CUDA pays this once
threepp scene + PhysX GPU + 2–5k particles	~0.3–1 GB	Grows with particle count and parallel envs
cuRobo motion generation	~1–2.5 GB	Reduce seeds/batch to shrink it
ACT training, 128–224 px images, batch 8	~2–3.5 GB	Use fp16/bf16 + small batch
SmolVLA fine-tuning	tight on 4 GB	Freeze the vision-language part; use the cloud if it crashes

These are planning estimates, not guarantees. Measure your real numbers with `nvidia-smi` and write them in your log.

3.3 The hybrid plan (free)

- **Laptop:** scene building, cuRobo, demo generation (overnight), quick training sanity checks, evaluation, video rendering.
- **Kaggle Notebooks:** free NVIDIA T4 or P100 with 16 GB; weekly GPU quota of about 30 hours with sessions capped at roughly 12 hours (check your quota counter in your account). Requires phone verification. Use it for full ACT runs and SmolVLA fine-tuning.
- **Lightning AI:** free tier with 15 monthly credits that buy roughly a couple of dozen T4 GPU-hours (more on interruptible machines), plus a persistent Linux workspace with a terminal. Good backup when you need a real Linux GPU machine.

- **Glue:** code on GitHub, datasets and checkpoints on Hugging Face Hub, so every machine pulls the same files.

> **WATCH OUT · free tiers change**
> Quotas and free credits are set by the providers and change without notice. Before planning a long run, check the quota shown in your account, and always save checkpoints often so a session timeout costs minutes, not hours.

3.4 Laptop habits for long runs

- Plug in the charger and choose the performance power profile.
- Keep the laptop on a hard surface; long GPU jobs get hot.
- Keep `watch -n 1 nvidia-smi` open in a second terminal to see VRAM and temperature.
- When you hit *out of memory*, lower in this order: batch size → image size → particle count → number of parallel environments.
- Optional later: `sudo prime-select on-demand` + reboot moves the desktop to the Intel GPU so all 4 GB go to your work.

KNOWLEDGE

4. The software stack explained

This chapter explains each tool: what it is, why it is in the project, the few ideas you must understand to use it, and where to learn more. Install commands are in Phase 0 (Chapter 8).

4.1 The base: Ubuntu, NVIDIA driver, CUDA

Three layers have to agree with each other before any GPU code runs:

Layer	What it is	Your status
NVIDIA driver	Kernel module that talks to the GPU hardware. Decides the maximum CUDA version you can use.	Done: 595.91.07 (proprietary), supports CUDA up to 13.2
CUDA runtime	Libraries programs use to run GPU code. PyTorch and Warp ship their own copy inside the pip package.	Comes with PyTorch
CUDA toolkit (nvcc)	The compiler for writing/building new CUDA code. Only needed when something compiles CUDA from source.	Install only if a build asks for nvcc

EXPLAINER · why “CUDA Version: 13.2” in nvidia-smi is not what you installed

The number at the top right of `nvidia-smi` is the **highest** CUDA version the driver can serve, not an installed toolkit. A PyTorch built for CUDA 12.x runs fine on a 13.2-capable driver, because newer drivers run older CUDA programs.

4.2 Conda environments

Conda gives you isolated Python installations. Your project lives in an environment called **gardener** (Python 3.10). Anything you pip-install while it is active stays inside it, so a broken experiment never breaks your system Python.

```
conda activate gardener      # run this in every new terminal
conda deactivate             # leave the environment
conda env export > environment.yml   # snapshot your environment (commit it to git)
```

4.3 PyTorch

The deep-learning framework used by cuRobo, LeRobot, ACT and SmolVLA. You need to be comfortable with tensors, moving data to the GPU (`.to('cuda')`), `nn.Module`, optimizers and a basic training loop. The official “Learn the Basics” tutorial covers exactly this in an afternoon.

4.4 threepp

threepp is a cross-platform C++20 3D library that copies the high-level API of three.js (a very popular JavaScript 3D library), so a scene is made of a Scene, Meshes, Materials, Cameras and Lights. Recent releases have pushed it towards a simulation platform. The features that matter for this project:

- **Python bindings on PyPI** (`pip install threepp`): scene graph, headless render-to-NumPy (camera images as arrays for your dataset), PhysX bindings, and `threepp.rl`, a GPU-vectorised RL stack (GpuSim + VecTask + PPO) with no external RL library needed.

- **PhysX integration:** rigid bodies, articulations (robots), joint drives and proprioceptive sensors.
- **Rendering:** an OpenGL backend and a Vulkan deferred renderer used for high-quality and ray-traced output.
- **threepp_editor:** a scene editor with a Play mode backed by PhysX; scenes are saved as plain three.js JSON.
- **Warp-based modules** are appearing (e.g. a granular-soil simulator), showing how Warp and threepp are combined.

WATCH OUT · CPU wheel vs GPU build

The pip wheel is the quickest way to start, and it ships the renderer and **CPU** PhysX. The GPU-vectorised parts (GPU PhysX, threepp.rl training) need a **PhysX GPU build** made from source with CMake. Plan: start with the pip wheel for weeks 1–3, and build from source only when you actually need GPU particles or GPU RL. Read the repo's build docs and release notes first; the project is moving fast (several releases in September 2026 alone).

4.5 NVIDIA PhysX

PhysX is NVIDIA's open-source physics engine (also inside Isaac Sim and many games). In this project it simulates:

- **Articulations** — iCub as a tree of links connected by joints, simulated with a stable reduced-coordinate solver (much better for robots than chaining separate joints).
- **Joint drives** — a spring-damper per joint that pulls it towards a target position (see 5.5).
- **Contacts and friction** — the can stays in the hand only because contact forces and friction hold it.
- **Particles** — PhysX has a position-based-dynamics (PBD) particle system for fluids on the GPU (see 5.7).

EXPLAINER · drive limits are impulses unless told otherwise

A threepp release note warns that PhysX treats a drive's max force as an impulse per substep by default, which can silently make joints hundreds of times stronger than intended. threepp now exposes a `drive_limits_are_forces` option in Python. When iCub's fingers crush or launch the can, check this setting first.

4.6 NVIDIA Warp

Warp is a Python framework where you write normal-looking Python functions, mark them as kernels, and Warp compiles them to fast CUDA (or CPU) code. Kernels can also be differentiable. Typical uses here:

- Counting how many water particles are inside each pot's volume, every step, on the GPU.
- Writing a custom particle-water model if PhysX particles are not exposed the way you need.
- Computing reward or success signals for thousands of particles without leaving the GPU.

```
import warp as wp

@wp.kernel
def count_in_pot(pos: wp.array(dtype=wp.vec3), c: wp.vec3, r: float, h: float,
                 out: wp.array(dtype=int)):
    i = wp.tid()
    p = pos[i]
    dx = p[0] - c[0]; dz = p[2] - c[2]
    if dx*dx + dz*dz < r*r and p[1] > c[1] and p[1] < c[1] + h:
        wp.atomic_add(out, 0, 1)
```

This is a sketch (it assumes y is up); adapt the axis convention to your scene.

4.7 NVIDIA cuRobo

cuRobo is a GPU-accelerated motion-generation library: given the robot, the obstacles and a goal pose for the hand, it finds a smooth, collision-free joint trajectory in tens of milliseconds by running many optimisations in parallel. Requirements in the current docs: Linux (Ubuntu 22.04 tested), an NVIDIA GPU newer than Turing with at least 4 GB VRAM, driver 580.65.06 or newer, and Python 3.10+. Your RTX 3050 (Ampere) with driver 595 meets all of them, at the minimum on memory.

- **Robot config:** a YAML file pointing to the URDF plus collision spheres for each link. iCub has no ready-made config, so you will create one (Phase 3).
- **World config:** obstacles as cuboids, cylinders or meshes (table, pots, can).
- **MotionGen:** the planner. You call it with a start joint state and a goal end-effector pose.
- **Memory knobs:** number of seeds, batch size, trajectory length — reduce them on 4 GB.

EXPLAINER · why cuRobo plans the arm but not the fingers

cuRobo plans a path for a kinematic chain to reach a goal pose while avoiding collisions. Grasping is different: the goal is to *make* contact with the right forces. So the arm goes to a pre-grasp pose with cuRobo, then your own script closes the fingers until they touch the handle. This split is standard practice.

4.8 Hugging Face LeRobot

LeRobot is an open-source PyTorch library for end-to-end robot learning: a standard dataset format, data loading and streaming, training scripts, and many policies (ACT, Diffusion Policy, π0, SmolVLA and others). You will use it for three things:

- **LeRobotDataset** — saving your demos in a format every policy understands (see 5.13).
- **Training** — one command to train ACT or fine-tune SmolVLA on your dataset.
- **Inference** — loading a trained policy and calling it each control step in your sim.

WATCH OUT · LeRobot changes fast

Command names, folder layouts and dataset versions have changed several times. Always follow the README and docs of the version you install, and pin that version in your environment file.

4.9 The iCub models package

The official URDF/SDF models of iCub live in the **robotology/icub-models** repository and are installable with conda (`conda install -c conda-forge icub-models`) or pip (`pip install icub-models`, which also gives a helper to resolve the model path). Chapter 6 explains which model variant to use.

4.10 Git, GitHub and Hugging Face Hub

- **Git/GitHub:** version control for code and configs. Commit small and often; never commit datasets or checkpoints.
- **Hugging Face Hub:** free hosting for datasets and model checkpoints. Kaggle and Lightning pull from it directly.
- **Git LFS:** large-file support some repos (including cuRobo historically) need: `sudo apt install git-lfs`.

KNOWLEDGE

5. Knowledge primer

Every concept the project uses, explained from scratch. Each section ends with what you need to be able to *do*, so you can test yourself.

5.1 Linux and the terminal

You will live in the terminal. The essentials:

Command	Meaning
pwd, ls -la, cd dir	Where am I, what is here, go somewhere
mkdir -p a/b, cp, mv, rm -r	Create, copy, move, delete (rm has no recycle bin)
sudo cmd	Run as administrator (asks your password; typing shows nothing)
sudo apt update; sudo apt install pkg	Refresh package lists; install a system package
nano file / code .	Edit a file in the terminal / open VS Code here
cmd | grep word	Filter output lines containing a word
Ctrl+C, Ctrl+Alt+T, Tab	Stop a program; new terminal; auto-complete names
htop, nvidia-smi, df -h	CPU/RAM use; GPU use; disk space
tmux	Keep long jobs running after you close the terminal

TIP · tmux for overnight jobs

`tmux new -s gen` starts a session; run your data generation; press `Ctrl+B` then `D` to detach. Tomorrow, `tmux attach -t gen` brings it back, even if the terminal was closed.

You should be able to: navigate folders, install packages, edit a config file, and keep a job running overnight.

5.2 GPUs, CUDA and memory

A GPU runs thousands of small threads at once. That is perfect for physics (every particle is updated in parallel), rendering (every pixel) and neural networks (big matrix multiplications). CUDA is NVIDIA's programming platform for this. Key ideas:

- **Host vs device:** host = CPU + RAM, device = GPU + VRAM. Copying between them is slow, so keep data on the GPU.
- **Kernels:** functions that run on the GPU, one thread per element.
- **Asynchronous execution:** GPU work is queued; timing code needs synchronisation to be honest.
- **Memory:** each process has its own CUDA context; two programs on the GPU both pay overhead.

You should be able to: read nvidia-smi, explain why the pipeline is staged, and fix an out-of-memory error.

5.3 Describing a robot: URDF

A URDF (Unified Robot Description Format) file is XML that describes a robot as a **tree**:

- **Links** are rigid parts (upper arm, forearm, finger phalanx) with a visual mesh, a collision shape, mass and inertia.
- **Joints** connect a parent link to a child link: *revolute* (rotates within limits), *continuous*, *prismatic* (slides) or *fixed*.
- Each joint has an origin (where the child frame sits relative to the parent), an axis, limits, and sometimes damping and friction.
- **Degrees of freedom (DoF)** = number of movable joints you can control.

```
<joint name="r_elbow" type="revolute">
  <parent link="r_upper_arm"/>
  <child  link="r_forearm"/>
  <origin xyz="0 0 -0.15" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit lower="0.26" upper="1.85" effort="10" velocity="3"/>
</joint>
```

Illustrative snippet; the real iCub names and numbers differ.

EXPLAINER · coupled joints (mimic) and why hands are tricky

Robot hands often have fewer motors than joints: one tendon or motor moves two or three finger segments together. URDF expresses this with `<mimic>` tags. Simulators handle mimic joints differently (some ignore them), so check how iCub's finger joints behave in PhysX: you may need to drive the coupled joints yourself with the same command.

You should be able to: open the iCub URDF, list the right-arm and right-hand joints with their limits, and draw the tree from torso to fingertip.

5.4 Kinematics

- **Joint space vs task space:** joint space is the list of joint angles; task space is where the hand is (position + orientation) in the world.
- **Forward kinematics (FK):** joint angles → hand pose. Always easy and unique.
- **Inverse kinematics (IK):** desired hand pose → joint angles. Can have zero, one or infinitely many solutions; solved numerically.
- **Redundancy:** iCub's arm has 7 joints for a 6-DoF pose, so many arm postures reach the same hand pose; planners pick one that avoids obstacles and limits.
- **Jacobian:** the matrix that maps small joint changes to small hand movements. IK and controllers use it.
- **Orientations:** stored as quaternions (4 numbers) to avoid gimbal lock; you will often see (w, x, y, z) vs (x, y, z, w) — check which order each library uses.

WATCH OUT · frames and units

Most bugs in robotics are frame bugs: the can's pose expressed in the world frame given to a function expecting the robot base frame, or y-up vs z-up, or degrees vs radians. Write the frame in every variable name: `can_pos_world`, `grasp_pose_base`.

5.5 Controlling joints: PD control

In simulation each joint usually has a **drive** that behaves like a spring and a damper:

```
torque = stiffness * (target_pos - pos) + damping * (target_vel - vel)
```

- High stiffness = stiff, precise joint, but can be unstable and violent on contact.
- High damping = smooth but sluggish. Too little damping = oscillation (jitter).
- Fingers need lower stiffness than the shoulder, otherwise grasps crush or eject objects.
- Policies usually output **target joint positions**; the drive turns them into torques.

You should be able to: tune an arm joint so it reaches a target in about half a second without overshoot or vibration.

5.6 Rigid-body simulation and contacts

- **Time step:** physics advances in small steps (e.g. 1/120 s or smaller). Smaller = more stable, slower.
- **Substeps / solver iterations:** extra internal iterations per step to make contacts and joints more accurate.
- **Collision shapes:** simplified geometry used for contacts (boxes, capsules, convex hulls). Visual meshes are only for drawing.
- **Friction:** the Coulomb model says the friction force can be at most μ × normal force. A held object slips when the tangential force (gravity, inertia while pouring) exceeds what friction can provide.
- **Contact offset / rest offset:** small distances that control when contacts start; wrong values cause floating or sinking.
- **Mass and inertia:** unrealistic masses (a 5 kg finger, a 1 g can) make the solver unstable. Keep mass ratios reasonable.

EXPLAINER · why friction-only holding is harder than welding

Many demos attach the object to the gripper once it is touched, which makes grasping trivial. Here the can must stay in the hand because the fingers squeeze it and friction resists gravity and the pouring motion. While pouring, the water inside moves, shifting the load, so the grip must stay firm. Practical recipe: a handle shape the fingers can wrap around, friction around 0.8–1.2 on fingers and handle, enough finger force, small finger stiffness and good damping.

5.7 Simulating water with particles

Instead of solving fluid equations on a grid, the water is a set of small spheres (particles) that push each other apart and stick together a little. Two families exist:

Method	Idea	Trade-off
SPH (Smoothed Particle Hydrodynamics)	Each particle computes density and pressure from its neighbours; pressure forces move it.	Physically grounded, but needs small time steps to stay stable.
PBD / PBF (Position-Based Fluids)	Particles are moved directly so that local density stays constant (a constraint), then velocities are derived.	Very stable at large steps, fast on GPU; less exact. This is what PhysX particles use.

- **Particle radius vs count:** the volume of water is fixed, so bigger particles mean fewer of them. Start with large particles (e.g. 5–8 mm for a toy can) and 2–5 thousand particles.
- **Cohesion, viscosity, surface tension:** parameters that make particles act more like water or honey. Tune by eye.
- **Measuring:** count particles whose positions fall inside each pot's cylinder (Warp kernel in 4.6). Success = fraction of poured particles that end up in the target pot.
- **Leakage:** if particles tunnel through thin walls, make walls thicker in the collision shape or reduce the time step.

WATCH OUT · sim-to-real gap for liquids

Particle water is an approximation: splashing, surface tension and flow rate will not match real water. That is acceptable for this project (the goal is a learning pipeline), but write it in your limitations section.

5.8 Motion planning

- **Configuration space:** the space of all joint angle combinations. A plan is a path through it that never hits an obstacle.
- **Sampling planners** (RRT, PRM) explore it randomly; **trajectory optimisation** starts from guesses and pushes them to be smooth, short and collision-free. cuRobo does both, in parallel on the GPU.
- **Collision spheres:** cuRobo approximates each robot link by a set of spheres because sphere-vs-world distance is very fast to compute. More spheres = more accurate, more memory.
- **Seeds:** parallel starting guesses for the optimiser. More seeds = higher success, more VRAM.
- **Time parameterisation:** turning a geometric path into a timed trajectory respecting velocity, acceleration and jerk limits.

TIP · building iCub collision spheres

cuRobo's documentation has a tutorial for configuring a new robot, including tools to fit spheres to link meshes. Start coarse (2–4 spheres per arm link), visualise them over the robot, and only add spheres where real collisions happen (forearm near the pot, hand near the can).

5.9 Grasping with a multi-fingered hand

- **Power grasp vs precision grasp:** wrapping the whole hand around a handle (power) vs holding with fingertips (precision). Use a power grasp for the can handle.
- **Pre-grasp pose:** a pose a few centimetres away from the handle, hand open, aligned. cuRobo plans to it; then a straight approach.
- **Grasp synergy:** instead of controlling every finger joint, use one number *s* from 0 (open) to 1 (closed) that maps to all finger joints at once. This is how humans' hands mostly move, and it makes learning far easier.
- **Close until contact:** increase *s* gradually; stop each finger when its contact force passes a threshold, then squeeze a little more.
- **Grasp check:** lift 2 cm and verify the can moved with the hand before continuing.

```
def synergy_targets(s, open_q, closed_q):
    # s in [0, 1]; open_q / closed_q are arrays of finger joint angles
    return open_q + s * (closed_q - open_q)
```

5.10 Cameras and rendering

- **Intrinsics:** focal length and image centre (how 3D points project to pixels). **Extrinsics:** where the camera is.
- **Rasterisation:** fast, standard real-time rendering. Use it for dataset images.
- **Ray tracing / path tracing:** physically based light simulation, beautiful but slow. Use it only for the showcase video.
- **Headless rendering:** rendering without opening a window, straight into a NumPy array. Essential for data generation.
- **Resolution:** VLAs and ACT commonly use roughly 224×224 or smaller inputs. Small images = smaller dataset, faster training, less VRAM.
- **Cameras to record:** iCub's eye camera (head view) and optionally a fixed scene camera. Two views help policies judge depth.

5.11 Imitation learning

Behaviour cloning (BC) is supervised learning: the dataset contains pairs (observation → expert action), and a network learns to predict the action from the observation. It is the simplest way to teach a robot from demonstrations.

EXPLAINER · the two classic problems of behaviour cloning

1. Compounding errors (distribution shift). The policy makes a small mistake, ends up in a state the expert never visited, has no idea what to do there, and the mistakes grow. Remedies: diverse demos (randomisation), demos that include small recoveries, predicting chunks of actions instead of one step.

2. Multimodality. If the expert sometimes goes left of the pot and sometimes right, a network trained with mean-squared error averages them and goes straight into the pot. Remedies: models that represent several possible actions (ACT's CVAE, diffusion policies).

ACT — Action Chunking with Transformers

- Introduced with the ALOHA low-cost bimanual robot (Zhao et al., 2023).
- Predicts a **chunk** of future actions (e.g. the next 50–100 steps) at once, which reduces compounding errors and makes motion smooth.
- Uses a transformer over image features (ResNet backbone) and joint states, trained as a CVAE to handle multimodality.
- **Temporal ensembling** optionally averages overlapping chunks at test time.
- Small (tens of millions of parameters), trains on a single consumer GPU: the right first policy for you.

Diffusion Policy

- Generates actions the way image diffusion models generate pictures: start from noise and denoise into an action sequence (Chi et al., 2023).
- Excellent with multimodal demos; slower at inference. A good second baseline if time allows.

> **TIP · data quality beats model size**
> With scripted demos, the most common failure is demos that are too similar. Randomise positions, water amount and lighting, add small noise to the expert's trajectory, and inspect a random sample of episodes by eye before training anything.

5.12 Vision-Language-Action models (VLAs)

A VLA takes **images + a text instruction + the robot state** and outputs **actions**. Most VLAs are built from a pretrained vision-language model (VLM) — the same kind of model that describes images — plus an **action expert**: a smaller network that turns the VLM's internal features into continuous robot actions.

Model	Size	Notes for you
SmolVLA (Hugging Face, 2025)	~450M parameters	Designed for affordable hardware; trained on community data; integrated in LeRobot; flow-matching action expert. Your main VLA.
OpenVLA (2024)	7B	Too large to fine-tune for free; good to read about.
π0 (Physical Intelligence, 2024)	~3B	Flow-matching action expert; influential design; too large for your budget.

- **Fine-tuning:** start from the pretrained weights and continue training on your dataset. The model keeps general visual/language knowledge and learns your robot and task.
- **Freezing:** keep the VLM backbone's weights fixed and train only the action expert. Much less memory, often enough for a single task family.
- **Memory tricks:** mixed precision (bf16/fp16), gradient checkpointing (recompute activations instead of storing them), gradient accumulation (simulate a bigger batch), and LoRA (train small low-rank adapters instead of full weights).
- **Embodiment mismatch:** SmolVLA was pretrained mostly on small arms with grippers, not a humanoid hand. Your action space (7 arm joints + 1–2 synergy values) is new, so the action expert must adapt — expect to need hundreds of demos, not tens.
- **Language grounding test:** keep the scene fixed and change only the instruction (“left plant” → “right plant”). If behaviour does not change, the model is ignoring language.

5.13 Robot datasets: the LeRobot format

- A dataset is a set of **episodes**; each episode is a sequence of **frames** at a fixed rate (fps).
- Each frame has **features**: e.g. `observation.images.eye`, `observation.state` (joint positions), `action`, plus the episode's **task** string (the instruction).
- Tabular data is stored as Parquet files; camera streams as MP4 videos (much smaller than PNGs); metadata (features, fps, statistics for normalisation) as JSON.
- Datasets are pushed to and streamed from the Hugging Face Hub, so Kaggle can load them with one line.
- Decide **fps** (e.g. 15–30), **action definition** (absolute joint targets) and **image size** before recording; changing them later means regenerating everything.

EXPLAINER · what “normalisation statistics” are

Joint angles, finger synergy and images live on very different scales. LeRobot computes the mean and standard deviation (or min/max) of every feature over the dataset and rescales inputs and outputs. If you mix datasets or change units, recompute them; wrong stats are a silent source of terrible policies.

5.14 Domain randomisation and sim-to-real

- **Domain randomisation:** vary everything that should not matter (positions, colours, lighting, camera pose slightly, friction, water amount) so the policy learns what *does* matter.
- **Held-out evaluation:** evaluate on seeds and, ideally, on combinations never seen in training.
- **Sim-to-real (future work):** transferring to a real robot needs realistic cameras, actuator models and robust policies. Mention it as a future step; it is out of scope for version 1.

5.15 Evaluating honestly

- Evaluate every policy on the **same** fixed list of scene seeds (e.g. 50).
- Report success rate **with a confidence interval**: with n = 50 trials, ±10–14 percentage points is normal. Do not over-interpret small differences.
- Define success before looking at results (Chapter 7).
- Report secondary metrics: water in target (%), spill (%), time to complete, failures by type (missed grasp, slipped can, wrong pot, timeout).
- Keep videos of a few successes **and** failures; failure videos are the most useful debugging tool.

5.16 Reinforcement learning (optional extension)

In RL the robot learns by trial and error from a reward instead of copying demos. threepp ships PPO (a standard RL algorithm) with GPU-vectorised environments, so a natural extension after version 1 is to **fine-tune the pour with RL**: reward water reaching the target level, penalise spills. RL needs many parallel environments, which is why it may need a bigger GPU (Lightning AI) rather than your laptop.

SPECIFICATION

6. The robot: iCub

iCub is an open-source humanoid robot developed at the Italian Institute of Technology (IIT), the size of a young child (about one metre tall), used by dozens of labs for research on manipulation, learning and cognition. It has around 53 degrees of freedom in total; the parts you use:

Part	Approx. DoF	Use in this project
Torso	3	Optional: lock it at first, unlock later to extend reach
Right arm	7 (shoulder 3, elbow 1, wrist 3)	Planned by cuRobo; policy outputs 7 targets
Right hand	about 9 motors driving about 19 finger joints (coupled)	Controlled through 1–2 synergy values
Head and eyes	6	Fixed; the eye camera gives the robot's point of view
Legs	12	Not used (base fixed to the ground)

Numbers are approximate and vary between iCub versions; take the exact joint list from the URDF you load.

6.1 Which model file to use

The **robotology/icub-models** repository contains many variants named after real robots in different labs. The one built for exactly this kind of work is **iCubGazeboV2_5_visuomanip**: version 2.5.5 *with hands and eyes, base link fixed to the ground and legs disabled*. Recent releases also added hands and cameras to the iCubGazeboV2_10 and iCubStanford01 models.

```
pip install icub-models resolve-robotics-uri-py
python - <<'EOF'
import resolve_robotics_uri_py as r
print(r.resolve_robotics_uri("package://iCub/robots/iCubGazeboV2_5_visuomanip/model.urdf"))
EOF
```

If that exact URI does not resolve, list the installed models folder and pick the *visuomanip* variant; the repository README lists the current package paths.

WATCH OUT · things to check right after loading

- Finger joints are revolute (movable), not fixed.
- How coupled finger joints are expressed (mimic tags) and whether PhysX honours them.
- Collision meshes of the hand: very detailed meshes make contact slow and unstable; replace with convex hulls or capsules if needed.
- Link masses: tiny finger masses with large torso masses can destabilise the solver; adjust for simulation if needed and note it.
- Units: URDF uses metres, kilograms and radians.

6.2 What you build around iCub

- **Reduced robot:** load the full URDF but disable/lock everything except torso (optional), right arm and right hand. Fewer moving parts = more stable and faster.
- **cuRobo config:** a YAML with base link, end-effector link (the right palm), the 7 arm joints, locked joints, and collision spheres for torso, arm and hand.
- **Hand synergy table:** for each finger joint, its “open” and “closed” angle (from the URDF limits, tuned by eye).
- **Grasp frame:** a frame attached to the palm that should align with the handle at grasp time.

SPECIFICATION

7. Task specification: the Little Gardener

7.1 The scene

Object	Description	Notes
Table	Flat box in front of iCub at waist height	Static collider
Watering can	Toy-sized (iCub's hand is small): body ~8–10 cm tall, side handle a finger-sized bar, spout	Dynamic rigid body; light (~100–200 g); built from convex parts
Water	2,000–5,000 particles inside the can	Amount randomised (40–90% full)
Plant pots ×2 (later ×3)	Cylinders with an open top, small plant mesh for looks	Static; the open top volume is the target
Cameras	iCub eye camera + one fixed scene camera	Low-res raster for data; ray tracing for video

7.2 The episode, step by step

1. **Reset:** sample a seed → place can, pots, set water amount, target pot, instruction.
2. **Reach:** cuRobo moves the arm to the pre-grasp pose next to the handle (hand open).
3. **Approach:** short straight motion so the palm meets the handle.
4. **Grasp:** close the synergy until contact, squeeze; lift 2–3 cm and check the can follows.
5. **Transport:** cuRobo moves the can above the target pot (spout over the opening), keeping it upright.
6. **Pour:** rotate the wrist to tilt the can; hold until the requested amount has left; tilt back.
7. **Return:** move back and place the can on the table; open the hand; retract.
8. **End:** stop when the hand has retracted, or at a timeout (e.g. 30 s).

7.3 Instructions (language)

Template	Effect on the expert
“Water the left plant.” / “Water the right plant.”	Chooses the target pot
“Give the left plant just a little water.”	Pour duration short (target ~20–30% of the can)
“Give the right plant a lot of water.”	Pour long (target ~60–80%)
“Water both plants.” (stretch goal)	Two pours in sequence
Paraphrases: “Please water the plant on your right”, “The left one needs water”	Same as the base template; tests language robustness

7.4 Randomisation ranges (starting values)

Parameter	Range	Why
Can position on table	±8 cm in x and y around a nominal spot	Forces visual localisation
Can yaw (rotation)	±25° (handle stays reachable)	Varies the grasp approach
Pot positions	±5 cm each, left/right order fixed in v1	Varies the pour target

Parameter	Range	Why
Water amount	40–90% of can volume	Varies pour time
Lighting	Intensity ±30%, light direction varied	Visual robustness
Object colours	Pot and can colour from a small palette	Prevents colour shortcuts
Expert noise	Small random offsets on waypoints	More diverse demos, recovery behaviour

7.5 Observation and action spaces

Signal	Content	Shape (example)
observation.images.eye	Eye camera RGB	3 × 224 × 224 (or 128)
observation.images.scene	Fixed camera RGB (optional)	3 × 224 × 224
observation.state	7 arm joint positions + synergy value(s) (+ torso)	8–11
action	Target positions for the same joints at the next step	8–11
task	Instruction string	text

> **TIP · keep the action space small**
> Learning 7 arm targets + 1 grasp value is far easier than 7 + 19 finger joints. If you later need a precise finger action, add a second synergy value (e.g. thumb opposition), not all joints.

7.6 Success metrics

Metric	Definition
Success	≥ 70% of poured particles land in the **instructed** pot, the can ends upright on the table, and the episode finishes before timeout
Amount accuracy (stretch)	Poured amount within the band of the instruction (little / a lot)
Water in target	Particles in target pot ÷ particles that left the can
Spill	Particles on table or floor ÷ particles that left the can
Wrong pot	More particles in the non-instructed pot than in the target
Grasp failure	Can never lifted, or dropped during transport/pour

Thresholds are starting points; fix them before evaluating any policy and do not change them afterwards.

7.7 Dataset target

- 200–500 successful expert episodes for ACT (start with 200, check results, add more if needed).
- Balanced: equal numbers per target pot and per amount level.
- 15–30 fps, episodes of 10–30 s → roughly 150–900 frames each.
- A separate list of 50 evaluation seeds never used in training.

EXECUTION

8. Step-by-step build plan

Seven phases. Each has a goal, steps, a “done when” checklist and the traps that usually appear. Before each phase, read the primer sections listed under *Read first*.

Phase 0 — Environment setup (week 1)

> **STATUS · where you are (2 October 2026)**
> Ubuntu 22.04 running on kernel 6.8.0-138. NVIDIA proprietary driver 595.91.07 working (`nvidia-smi` shows the RTX 3050 with 4096 MiB). Miniconda installed, environment **gardener** (Python 3.10) created, PyTorch installing.

Read first: 4.1–4.3, 5.1–5.2.

Step 0.1 — PyTorch check

```
conda activate gardener
pip install torch torchvision
python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available(), torch.cuda.get_
# expected: ... True NVIDIA GeForce RTX 3050 Laptop GPU
```

Step 0.2 — System tools

```
sudo apt install -y git git-lfs build-essential cmake ninja-build tmux htop
git lfs install
```

Step 0.3 — Warp

```
pip install warp-lang
python -c "import warp as wp; wp.init()"      # should list your CUDA device
python -m warp.examples.browse                 # optional: browse the examples
```

Step 0.4 — threepp (Python wheel first)

```
pip install threepp
python -c "import threepp as tp; s = tp.Scene(); print('threepp OK')"
# Later, only if you need GPU PhysX / threepp.rl, build from source:
git clone https://github.com/markaren/threepp.git   # follow its build docs
```

Step 0.5 — cuRobo

```
git clone https://github.com/NVlabs/curobo && cd curobo
python -c "import torch; print(torch.version.cuda)"   # 12.x or 13.x ?
pip install -e ".[cu12]"     # if PyTorch reports CUDA 12.x
# pip install -e ".[cu13]"   # if PyTorch reports CUDA 13.x
python -c "import curobo; print(curobo.__version__)"
cd ..
```

These extras follow the current cuRobo install guide, which recommends the *uv* tool; plain pip inside conda is usually fine. If the build complains that nvcc is missing, install a CUDA toolkit that matches PyTorch's CUDA version (for example `conda install -c nvidia cuda-toolkit=12.8`) and retry.

Step 0.6 — iCub model and LeRobot

```
pip install icub-models resolve-robotics-uri-py
pip install lerobot          # or install from source following the LeRobot README
```

WATCH OUT · dependency clashes

LeRobot may want a different PyTorch version than cuRobo. If installing it changes your torch version and breaks cuRobo, create a second environment **gardener-train** for LeRobot. The staged pipeline makes this painless: simulation/planning in one env, training in the other, files in between.

Step 0.7 — Run the original demo (sanity test)

Open the code link from the original LinkedIn post, clone it, and run it with a reduced particle count. If it runs, your whole stack works on this machine. Write down every problem you hit in **docs/install_notes.md**.

Done when

- torch, warp, threepp, curobo, icub-models and lerobot all import without error.
- A cuRobo example (e.g. motion generation for its demo robot) runs on your GPU.
- `conda env export > environment.yml` committed to your new GitHub repo.

Phase 1 — iCub in the simulator (week 2)

Read first: 5.3–5.6, Chapter 6.

1. Resolve and load the *visuomanip* URDF into a threepp + PhysX scene with the base fixed.
2. Print all joints with limits; make a table of right-arm and right-hand joints in your notes.
3. Lock legs, head and left arm. Set drive stiffness/damping per joint group (shoulder, elbow, wrist, fingers).
4. Write a small test: move each arm joint through its range one by one; check direction and limits.
5. Implement the hand synergy (5.9) and test open/close in the air.
6. Add the eye camera and save one image as PNG.
7. Record a short clip: iCub waving and opening/closing its hand — your first LinkedIn-able result.

Done when: iCub stands still without jitter for 60 s; every arm joint tracks a target; the hand opens and closes smoothly; you get camera images as NumPy arrays.

Traps: exploding joints (masses/inertias or stiffness too high), fingers passing through each other (collision filtering), mimic joints ignored.

Phase 2 — The garden scene and water (week 3)

Read first: 5.6–5.7, Chapter 7.1.

1. Build the table and pots from simple shapes; place them within iCub's reach (test with FK/IK).
2. Model the watering can from convex parts (body, handle bar, spout). Give it realistic mass and friction.
3. Fill the can with particles; tune radius, cohesion and viscosity until it sloshes and pours plausibly.
4. Write the particle counter (Warp kernel) for each pot and the table.
5. Test pouring *without* the robot: script the can's pose to tilt over a pot and check the counts.

6. Measure VRAM and simulation speed at 2k, 5k, 10k particles; pick the budget.

Done when: a kinematically moved can pours into a pot, counts are correct, and water does not leak through walls.

Phase 3 — The scripted expert and demo generation (weeks 4–5)

Read first: 5.8–5.9, 7.2–7.4. This is the longest and hardest phase.

3a. cuRobo for iCub

1. Create the robot config YAML: URDF path, base link, end-effector link (right palm), the 7 arm joints, locked joints.
2. Fit collision spheres for torso, upper arm, forearm and hand; visualise and adjust.
3. Create the world config (table, pots, can as simple shapes).
4. Plan from home to a hand-made goal pose; replay the trajectory in threepp and check it is collision-free.
5. Reduce seeds/batch until peak VRAM stays below ~2.5 GB.

3b. The expert

1. Write the state machine of 7.2 (reach → approach → grasp → lift check → transport → pour → return).
2. Compute pre-grasp, pour and place poses from the scene's object poses (with the noise from 7.4).
3. Plan all arm segments with cuRobo first and save them to `data/plans/` (planning stage).
4. Replay them in the physics sim with the hand script, logging everything (recording stage).
5. Mark an episode successful only if it passes the metric in 7.6; discard or log failures.

3c. Mass generation

1. Turn recording into the LeRobot format (5.13): eye (+ scene) images, state, action, task text.
2. Run overnight in tmux; make the script skip episodes that already exist so it resumes after a crash.
3. Inspect 10 random episodes visually, check action/state plots, then push the dataset to the Hub.

Done when: the scripted expert succeeds on ≥ 80% of random scenes and 200+ good episodes are on the Hub. **Milestone video:** the scripted gardener.

> **WATCH OUT · if grasping does not work after a week**
> Simplify instead of getting stuck: make the handle a thicker cylinder, raise finger and handle friction, lower the water amount (lighter can), or use a mug-style closed handle. Write down what you changed; it is a legitimate engineering decision.

Phase 4 — Imitation learning with ACT (week 6)

Read first: 5.11, 5.13, 5.15.

1. Train ACT for a few hundred steps on the laptop (small images, batch 8, mixed precision) to check that the loss goes down and nothing crashes.
2. Run the full training on Kaggle: load the dataset from the Hub, save checkpoints every N steps to the Kaggle output folder, upload the final checkpoint to the Hub.
3. Write the evaluation script: load the policy, run it in the sim on the 50 evaluation seeds, save metrics and videos.
4. Compare with the scripted expert on the same seeds; analyse failures by type.

Done when: a results table (expert vs ACT) with confidence intervals exists, plus success and failure videos.

Phase 5 — Fine-tuning SmolVLA (weeks 7–8)

Read first: 5.12.

1. Read the SmolVLA model card and the LeRobot SmolVLA docs; start from the pretrained base checkpoint.
2. Fine-tune on Kaggle with your dataset (mixed precision, small batch, gradient accumulation; freeze the VLM if memory is short).
3. Evaluate on the same 50 seeds. Then run the **language test**: identical scenes, only the instruction changes (left/right, little/a lot).
4. Optionally try inference on the laptop (inference needs much less memory than training).

Done when: SmolVLA's success rate and instruction-following accuracy are in the results table.

Phase 6 — Showcase (week 9)

1. Render the best episodes with ray tracing, offline, frame by frame at modest resolution; assemble with ffmpeg.
2. Write the README: motivation, demo GIF/video, architecture figure, install, how to reproduce, results table, limitations, future work.
3. Publish the dataset card and model cards on the Hub.
4. Write the post (Chapter 13).

```
ffmpeg -framerate 30 -i frames/%05d.png -c:v libx264 -pix_fmt yuv420p -crf 18 gardener.mp4
```

EXECUTION

9. Schedule

Based on about 5 hours a day, six days a week (30–35 h/week), starting 2 October 2026.

Week	Dates	Phase	Goal / milestone
1	Oct 2 – Oct 9	0	Stack installed, original demo runs
2	Oct 10 – Oct 16	1	iCub moving arm and fingers stably — first clip
3	Oct 17 – Oct 23	2	Garden scene with pouring water and particle counts
4–5	Oct 24 – Nov 6	3	**Scripted gardener video** + 200+ demos on the Hub (key checkpoint)
6	Nov 7 – Nov 13	4	ACT trained and evaluated
7–8	Nov 14 – Nov 27	5	SmolVLA fine-tuned, language test done
9	Nov 28 – Dec 4	6 / buffer	Video, README, post; absorbs earlier delays

9.1 A typical 5-hour day

Block	Time	Content
Plan	10 min	Read yesterday's log; write today's 1–3 goals
Learn	45 min	The primer section for today's work, or a paper/tutorial
Build	3 h (two blocks with a break)	Implementation and debugging
Run	30 min	Start overnight jobs (generation/training), check yesterday's results
Log	15 min	What worked, what broke, numbers measured, next step

TIP · protect the pace

Take one full day off per week. When stuck on the same bug for more than two hours, write the problem down precisely, switch to another task, and come back the next day — or simplify.

EXECUTION

10. Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Install/driver problems eat days	Medium (already happened once)	Medium	Follow Chapter 12; keep the old kernel in GRUB; snapshot environment.yml
4 GB VRAM too small for a stage	High	Medium	Staged pipeline; reduce particles/batch/image size; Kaggle or Lightning AI
iCub URDF unstable in PhysX (hands)	Medium	High	Simplify collision meshes; tune masses; drive coupled joints yourself
Friction-only grasp keeps slipping	High	High	Thicker handle, higher friction, lighter can, firmer synergy; fall back to a mug handle
No ready cuRobo config for iCub	Certain	Medium	Budget 2–3 days; start with coarse spheres
Particle water leaks or is slow	Medium	Medium	Thicker walls, smaller step, fewer bigger particles
Demos too uniform → policy overfits	Medium	High	Randomisation + expert noise; inspect samples
SmolVLA does not fit / learns poorly	Medium	Medium	Freeze VLM, more demos, report honestly; ACT result still stands
Library API changes (threepp, LeRobot)	High	Low–Medium	Pin versions; read release notes before upgrading
Free GPU quota runs out	Medium	Low	Plan runs around the weekly reset; checkpoint often
Burnout at 5 h/day	Medium	High	Day off weekly; the week-9 buffer; celebrate milestones

EXECUTION

11. Repository and engineering practices

11.1 Suggested layout

```
icub-gardener/
├── README.md
├── environment.yml              # conda env snapshot (sim + planning)
├── environment-train.yml        # optional second env for LeRobot
├── configs/
│   ├── scene.yaml               # objects, randomisation ranges, particles
│   ├── icub_curobo.yml          # cuRobo robot config (spheres, joints)
│   └── expert.yaml              # waypoint offsets, pour durations
├── gardener/                    # python package
│   ├── sim/        robot.py  scene.py  water.py  camera.py
│   ├── expert/     planner.py  hand.py  state_machine.py
│   ├── data/       record.py  to_lerobot.py
│   └── eval/       evaluate.py  metrics.py
├── scripts/  plan_episodes.py  record_episodes.py  render_video.py
├── notebooks/  kaggle_train_act.ipynb  kaggle_finetune_smolvla.ipynb
├── docs/  install_notes.md  log.md  results.md
└── tests/  test_kinematics.py  test_particle_counter.py
```

11.2 Practices

- **Configs, not constants:** every number you tune lives in a YAML file; scripts take `--config` and `--seed`.
- **Seeds everywhere:** NumPy, Python random, PyTorch, and the scene randomiser.
- **Small commits** with messages that say why (“lower finger stiffness: can was ejected”).
- **.gitignore** data/, outputs/, frames/, *.npz, *.mp4 — those go to the Hub.
- **Tests for the tricky bits:** particle counter, frame conversions, synergy mapping.
- **Plots before training:** plot action and state curves of a few episodes; jumps and flat lines reveal recording bugs.

11.3 Daily log template (docs/log.md)

```
## 2026-10-03  (Phase 0, day 2)
Goal: install Warp + threepp + cuRobo
Done:
- warp OK (CUDA device found)
- curobo build failed: nvcc missing -> installed cuda-toolkit 12.8 -> OK
Numbers: cuRobo example peak VRAM 1.9 GB
Problems / open questions:
- threepp wheel has CPU PhysX only; GPU build needed later?
Next: run original demo with 2k particles
```

EXECUTION

12. Troubleshooting journal

Problems already met during setup on 2 October 2026, with causes and fixes. Keep adding to this chapter's equivalent in **docs/install_notes.md** — it becomes a valuable part of your README.

Symptom	Cause	Fix
`nvidia-smi`: couldn't communicate with the driver	Old driver 580 not matched to the kernel	`sudo ubuntu-drivers install` (installed 595-open + new kernel 6.8.0-138)
After reboot: touchpad and Wi-Fi dead; `i2c_hid_acpi` module not found	New kernel installed without its **linux-modules-extra** package (where touchpad/Wi-Fi drivers live)	USB tethering for internet, then `sudo apt install linux-modules-extra-6.8.0-138-generic linux-generic-hwe-22.04`
GRUB menu does not appear	Hidden menu with zero timeout	Set GRUB_TIMEOUT_STYLE=menu and GRUB_TIMEOUT=10 in /etc/default/grub, then `sudo update-grub`
Driver module loaded but `/dev/nvidia*` missing; log stops at “Loading driver”; `sudo nvidia-smi` hangs	The open kernel module (595-open) hung while initialising this laptop's GPU	Switch to proprietary: `sudo apt install nvidia-driver-595 linux-modules-nvidia-595-generic-hwe-22.04`, reboot
`conda: command not found` right after install	Shell not reloaded	`source ~/.bashrc` or open a new terminal
CondaToSNonInteractiveError	New conda requires accepting channel Terms of Service once	Run the two `conda tos accept ...` commands it prints
`Unable to locate package`	Typo in the package name	Copy-paste names; use Tab completion

12.1 Safety nets you now have

- GRUB menu shows for 10 s: you can boot the older kernel 6.8.0-111 or Windows if an update breaks something.
- Ctrl+Alt+F3 gives a text login if the graphical desktop fails.
- `nmcli device wifi connect "NAME" password "PASS"` connects Wi-Fi from the terminal.
- Secure Boot stays on: Ubuntu's pre-signed NVIDIA modules (*linux-signatures-nvidia*) are accepted without MOK enrolment.

12.2 Diagnostic commands

```
uname -r                                  # running kernel
nvidia-smi                                # driver + GPU status
lsmod | grep nvidia                       # driver modules loaded?
ls -l /dev/nvidia*                        # device files present?
sudo dmesg | grep -i -E "nvidia|NVRM" | tail -30   # driver messages
prime-select query                        # nvidia / on-demand / intel
mokutil --sb-state                        # Secure Boot on/off
dpkg -l | grep -E "nvidia-driver|linux-modules-nvidia"   # what's installed
python -c "import torch; print(torch.cuda.is_available())"
```

WATCH OUT · be careful with system updates

There are ~900 pending updates. Updating is fine, but a new kernel can again arrive without matching NVIDIA modules or extras. After any kernel update, check `nvidia-smi` and the touchpad before starting long jobs, and keep the previous kernel in GRUB.

EXECUTION

13. Showcase and career value

A project only helps your career if people can see and understand it in two minutes. Plan the presentation from the start.

13.1 What to publish

- **A 60–90 s video:** ray-traced scripted gardener, then the learned policy following two different instructions, then one honest failure.
- **README:** the one-sentence pitch (1.2), a GIF at the top, the pipeline figure, results table, limitations, how to reproduce on a 4 GB GPU.
- **Dataset and model cards on the Hub** — other people can use your data, which is rare for liquid tasks.
- **A short technical write-up** (blog or post): what was hard (friction grasp, particles on 4 GB, iCub cuRobo config) and what you learned.

13.2 Results table template

Policy	Success (50 scenes)	Water in target	Spill	Language test	Train compute
Scripted expert	…% ± …	…%	…%	n/a	—
ACT	…% ± …	…%	…%	n/a (single task)	Kaggle T4, … h
SmolVLA (fine-tuned)	…% ± …	…%	…%	…% correct pot	Kaggle T4, … h

13.3 Skills this project demonstrates

- Robot modelling and simulation (URDF, PhysX articulations, contacts, particles).
- GPU programming (Warp kernels) and working under tight memory limits.
- Motion planning integration (cuRobo config for a new robot).
- Data engineering for robot learning (automatic demo generation, LeRobot datasets).
- Training and evaluating modern policies (ACT, VLA fine-tuning) with honest metrics.
- Linux system administration (drivers, kernels, Secure Boot) — it is real industry knowledge.

13.4 Possible extensions (version 2)

- RL fine-tuning of the pour with threepp.rl (on Lightning AI).
- Estimate fill level from images (computer vision head) instead of reading particles.
- Both hands: hold the pot with the left hand while pouring.
- More tasks with the same scene: stir, fill to a line, wipe a spill.
- A cheap real arm (e.g. SO-101) for a first sim-to-real experiment.

REFERENCE

Appendix A — Glossary

Term	Meaning
ACT	Action Chunking with Transformers: imitation-learning policy that predicts chunks of future actions
Action space	What the policy outputs each step (here: joint position targets + grasp synergy)
Articulation	A robot simulated as a tree of links and joints with a dedicated stable solver
Behaviour cloning	Supervised learning from (observation, expert action) pairs
bf16 / fp16	16-bit number formats that halve memory and speed up training (mixed precision)
Collision sphere	Sphere approximating part of a link for fast collision checks in cuRobo
CUDA	NVIDIA's platform for running general computations on the GPU
CVAE	Conditional variational autoencoder: lets ACT represent several valid ways to act
Distribution shift	The policy reaches states unlike the training data, so its errors compound
DoF	Degree of freedom: one independent movable joint
Domain randomisation	Varying irrelevant scene properties so policies generalise
Drive (joint)	Spring-damper that pulls a joint towards a target position/velocity
End-effector	The part of the robot that acts on the world (here: the right palm)
Episode	One complete attempt at the task from reset to end
Fine-tuning	Continuing to train a pretrained model on your own data
Flow matching	Generative technique (related to diffusion) used by π0/SmolVLA action experts
FK / IK	Forward / inverse kinematics: joints to hand pose / hand pose to joints
Gradient accumulation	Summing gradients over several small batches to mimic a big batch
Gradient checkpointing	Recomputing activations during backprop to save memory
Headless rendering	Rendering images without opening a window
Kernel (GPU)	A function executed in parallel by many GPU threads
Kernel (Linux)	The core of the operating system; drivers are kernel modules
LeRobot	Hugging Face's library for robot-learning datasets, training and policies
LoRA	Low-rank adapters: small trainable matrices added to a frozen big model
Mimic joint	A joint whose angle is tied to another joint (coupled fingers)
MOK	Machine Owner Key: lets Secure Boot accept self-signed kernel modules
PBD / PBF	Position-based dynamics / fluids: stable particle simulation methods
Policy	The function (network) mapping observations to actions
PPO	Proximal Policy Optimisation, a standard reinforcement-learning algorithm
Pre-grasp pose	Hand pose just before contact, from which a straight approach begins
Quaternion	4-number representation of a 3D rotation
Secure Boot	Firmware feature that only loads signed kernels/modules
SPH	Smoothed Particle Hydrodynamics, a classic particle fluid method

Term	Meaning
Synergy (hand)	Low-dimensional control of many finger joints with one or two values
Teleoperation	A human controlling the robot to record demonstrations
URDF	Unified Robot Description Format: XML file describing links and joints
VLA	Vision-Language-Action model: images + instruction in, robot actions out
VLM	Vision-Language Model: images + text in, text out (the backbone of most VLAs)
VRAM	Memory on the graphics card

REFERENCE

Appendix B — Resources and links

Links collected on 2 October 2026. Prefer official docs and the papers' own pages; if a link has moved, search the project name.

Core stack

- **threepp** — C++20 three.js-style 3D library with Python bindings, PhysX, RL tools. https://github.com/markaren/threepp
- **threepp on PyPI** — Python package page and binding docs. https://pypi.org/project/threepp/
- **threepp releases** — Read before upgrading; behaviour changes are listed here. https://github.com/markaren/threepp/releases
- **NVIDIA PhysX** — Open-source physics engine and its documentation. https://github.com/NVIDIA-Omniverse/PhysX
- **NVIDIA Warp** — Source, install notes and examples. https://github.com/NVIDIA/warp
- **Warp documentation** — User guide and API reference. https://nvidia.github.io/warp/
- **cuRobo** — Source code. https://github.com/NVlabs/curobo
- **cuRobo installation (current docs)** — Requirements and install commands. https://nvlabs.github.io/curobo/latest/getting-started/installation.html
- **cuRobo tutorials (classic site)** — Robot configuration, collision spheres, motion generation. https://curobo.org

Robot

- **icub-models** — Official URDF/SDF models of iCub; model table and usage from Python. https://github.com/robotology/icub-models
- **iCub project** — Background on the robot and its community. https://icub.iit.it
- **URDF tutorials (ROS)** — How links, joints and frames are written. https://wiki.ros.org/urdf/Tutorials

Learning: libraries and models

- **LeRobot** — Datasets, training scripts, ACT, Diffusion Policy, SmolVLA. https://github.com/huggingface/lerobot
- **LeRobot documentation** — Dataset format, training and evaluation guides. https://huggingface.co/docs/lerobot
- **SmolVLA base checkpoint** — Model card and weights to fine-tune. https://huggingface.co/lerobot/smolvla_base
- **PyTorch tutorials** — Learn the Basics: tensors, models, training loop. https://pytorch.org/tutorials/beginner/basics/intro.html

Papers (read in this order)

- **ACT / ALOHA — Zhao et al., 2023** — Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. https://arxiv.org/abs/2304.13705
- **Diffusion Policy — Chi et al., 2023** — Visuomotor policy learning via action diffusion. https://arxiv.org/abs/2303.04137
- **SmolVLA — Shukor et al., 2025** — A VLA for affordable and efficient robotics. https://arxiv.org/abs/2506.01844

- **LeRobot paper, 2026** — An open-source library for end-to-end robot learning. https://arxiv.org/abs/2602.22818
- **OpenVLA — Kim et al., 2024** — Open-source 7B VLA (background reading). https://arxiv.org/abs/2406.09246
- **π0 — Black et al., 2024** — Vision-language-action flow model (background reading). https://arxiv.org/abs/2410.24164
- **cuRobo — Sundaralingam et al., 2023** — CUDA-accelerated robot motion generation. https://arxiv.org/abs/2310.17274

For particle fluids, look up *"Position Based Fluids"* (Macklin and Müller, SIGGRAPH 2013) — the method family PhysX particles build on.

Compute

- **Kaggle notebooks docs** — Technical specs, GPU sessions and quotas. https://www.kaggle.com/docs/notebooks
- **Lightning AI** — Free tier with monthly credits and persistent Studios. https://lightning.ai/pricing
- **Hugging Face Hub** — Hosting datasets and checkpoints. https://huggingface.co/docs/hub

Courses for the background

- **Modern Robotics (Lynch and Park)** — Free textbook and videos: kinematics, dynamics, planning. https://hades.mech.northwestern.edu/index.php/Modern_Robotics
- **Hugging Face Robotics Course** — Hands-on robot learning with LeRobot. https://huggingface.co/learn
- **The Missing Semester (MIT)** — Shell, git, tmux, editors. https://missing.csail.mit.edu

> **TIP · the original demo**
> Add the link to the code from the LinkedIn post that inspired this project here and in your README credits. Reading its source is the fastest way to see how threepp, PhysX, Warp and cuRobo were wired together, especially how the water particles were done.

REFERENCE

Appendix C — Command cheat sheet

Every session

```
conda activate gardener
cd ~/icub-gardener && git pull
watch -n 1 nvidia-smi        # in a second terminal
```

Long jobs

```
tmux new -s gen             # start
python scripts/record_episodes.py --config configs/scene.yaml --start 0 --n 300
# Ctrl+B then D to detach;   tmux attach -t gen   to come back
```

Git

```
git status
git add -A && git commit -m "pour: lower tilt speed, fewer spills"
git push
```

Hugging Face Hub

```
pip install -U huggingface_hub
huggingface-cli login        # paste a write token from your HF settings
# then push datasets / checkpoints from your scripts
```

GPU and system

```
nvidia-smi                    # VRAM and temperature
sudo prime-select on-demand   # desktop on Intel GPU (reboot) -> frees VRAM
sudo prime-select nvidia      # back to NVIDIA for everything
df -h ~                       # disk space
htop                          # CPU and RAM
```

Video

```
ffmpeg -framerate 30 -i frames/%05d.png -c:v libx264 -pix_fmt yuv420p -crf 18 out.mp4
```

Good luck, and have fun watching your little gardener water its first plant.